CASE STUDY 3

Enhancing the Multi-Agent Conversational Assistant

A2A Communication | Semantic Memory | Langfuse Prompt as a Service | Evaluations

> **Challenge Summary:** Extend the existing Case Study 2 solution into a more collaborative, context-aware, observable, and measurable multi-agent system. Teams should enhance their current implementation rather than rebuild the project from scratch.

1. Background and Continuity from Case Study 2

In Case Study 2, teams developed a Multi-Agent Conversational Assistant using LangGraph and MCP. The solution included a Supervisor/Planner Agent, specialized Web Search and Text-to-SQL agents, a memory layer, and support for web, database, and hybrid queries.

Case Study 3 builds directly on that foundation. The goal is to upgrade the current application with four advanced capabilities: agent-to-agent collaboration, semantic memory, centralized prompt management through Langfuse, and a structured evaluation framework.

2. Case Study Objective

Enhance the existing Case Study 2 project so that it can:

- Enable autonomous and structured collaboration between agents using an A2A approach.
- Retain and retrieve meaningful long-term information using semantic memory.
- Externalize, version, and manage prompts through Langfuse Prompt as a Service.
- Measure response quality and system behaviour through repeatable evaluations.

3. Required Enhancements

3.1 A2A (Agent-to-Agent Communication)

A2A enables specialized agents to communicate and collaborate as independent participants, instead of routing every intermediate exchange through a single hard-coded function chain. In this case study, teams should introduce a clear mechanism for agents to discover capabilities, exchange task context, delegate work, and return structured results.

Expected enhancement:

- Define each agent's role, capability, accepted input, and expected output.
- Allow one agent to delegate a sub-task to another agent when additional expertise is required.
- Pass relevant context, task status, and results using a consistent message or payload structure.
- Handle agent failure, unavailable capability, invalid output, and timeout scenarios gracefully.
- Demonstrate at least one end-to-end query that requires collaboration between two or more specialized agents.

> **Example scenario:** A user asks, “Compare our internal sales performance with current external market trends.” The database agent retrieves internal sales data, the web agent collects external information, and an analysis or synthesis agent combines the findings into one response.

3.2 Semantic Memory

Semantic memory stores reusable knowledge about facts, concepts, user preferences, and prior information. Unlike a simple chat history, it retrieves information based on meaning and relevance, helping the assistant use important knowledge across interactions.

Expected enhancement:

- Identify what information should be stored as semantic memory, such as stable user preferences, validated facts, business definitions, or reusable prior findings.
- Generate embeddings and store memories in a suitable vector-enabled store.
- Retrieve relevant memories using semantic similarity before generating a response.
- Inject only relevant memory into the agent context to avoid unnecessary context growth.
- Include a strategy for duplicate memories, conflicting facts, outdated information, and memory updates.
- Demonstrate that a later conversation can correctly reuse information captured in an earlier interaction.

Teams should clearly distinguish semantic memory from short-term conversation state, response caching, and raw transcript storage.

3.3 Langfuse - Prompt as a Service

Prompt as a Service means prompts are managed outside the application code and retrieved at runtime. Langfuse should act as the centralized location for prompt templates, versions, labels, and variables, enabling teams to improve prompts without embedding every change directly in the codebase.

Expected enhancement:

- Move selected prompts, such as router, planner, SQL-generation, synthesis, or evaluator prompts, from source code into Langfuse.
- Use prompt variables for dynamic inputs instead of constructing unstructured prompt strings.
- Maintain prompt versions and identify which version is used in each execution trace.
- Use labels or an equivalent controlled approach to separate development and approved prompt versions.
- Add a fallback mechanism so the application can handle temporary prompt retrieval failure.
- Demonstrate a prompt update through Langfuse without changing the core application logic.

3.4 Evals (Evaluation Framework)

Evaluations provide evidence that the enhanced assistant is accurate, relevant, safe, reliable, and improving over time. Teams should create a repeatable evaluation approach rather than relying only on manual observation or a few successful demo queries.

Expected enhancement:

- Create a representative evaluation dataset containing normal, complex, hybrid, ambiguous, and failure-oriented test cases.
- Define clear evaluation criteria for the complete response and, where applicable, individual agent outputs.
- Include deterministic checks where possible, such as schema validity, required fields, tool selection, routing correctness, SQL execution success, and citation presence.
- Include model-based or rubric-based evaluation for qualities such as relevance, correctness, groundedness, completeness, and response clarity.
- Link evaluation results with traces, prompt versions, and relevant system metadata in Langfuse.
- Compare results before and after at least one prompt, routing, memory, or agent enhancement.
- Summarize failures and explain the improvement made based on evaluation evidence.

4. Integrated Workflow Expectation

1. A user submits a query through the existing application interface.
2. The Supervisor/Planner determines the required agents and task sequence.
3. Relevant semantic memories are retrieved and added to the working context.
4. Prompts are obtained from Langfuse using the selected prompt version or label.
5. Agents collaborate through the A2A mechanism and use existing MCP tools where required.
6. The final response is generated, while traces capture agent steps, prompt versions, tool calls, latency, and errors.
7. The execution is evaluated using the defined checks and evaluation criteria.

5. Minimum Functional Demonstrations

- A multi-agent query that uses A2A collaboration across at least two agents.
- A follow-up query that retrieves and applies previously stored semantic memory.
- A prompt retrieved from Langfuse, including visible version or label information in the trace.
- An evaluation run that scores a defined test dataset and explains at least one improvement.
- A controlled failure scenario showing fallback, error handling, or graceful degradation.

6. Deliverables

Deliverable	Required content	Suggested format
Enhanced source code	Working Case Study 2 solution extended with all four capabilities.	ZIP or repository link
Architecture and flow	Updated architecture showing LangGraph orchestration, MCP usage, A2A exchange, memory, Langfuse, and eval flow.	Diagram in PDF/PPT/Word
README / setup guide	Prerequisites, configuration, environment variables, run steps, components, and assumptions.	Markdown or Word

Deliverable	Required content	Suggested format
Demo evidence	Screenshots, traces, outputs, or a short recording covering the minimum demonstrations.	PPT/PDF/video
Evaluation report	Test dataset, metrics or rubrics, results, failure analysis, comparison, and improvements.	CSV/Excel plus summary
Presentation	Problem, design choices, implementation, demonstration, results, limitations, and next steps.	PPT or PDF

7. Evaluation Criteria

Area	What will be reviewed
A2A implementation	Clarity of agent contracts, meaningful collaboration, structured exchanges, and failure handling.
Semantic memory	Appropriate memory selection, retrieval quality, relevance, lifecycle handling, and correct reuse.
Prompt management	Externalized prompts, variables, version traceability, controlled promotion, and fallback.
Evaluation quality	Representative test set, suitable checks, traceability, comparable results, and actionable analysis.
Integration and architecture	How effectively the new capabilities extend the existing LangGraph and MCP solution.
Engineering quality	Modularity, configuration, logging, error handling, security awareness, and documentation.
Demonstration and explanation	Working end-to-end scenarios and clear explanation of decisions, limitations, and results.

8. Important Guidelines

- Build on the existing Case Study 2 project. Do not replace it with an unrelated application.
- Keep the solution modular so agents, prompts, memory components, and evaluators can be changed independently.
- Do not expose API keys, credentials, connection strings, or sensitive data in source code, screenshots, traces, or submissions.
- Use suitable mock or sample data when access to an external system is unavailable.
- Document assumptions and clearly identify any capability that is simulated rather than fully integrated.
- Prioritize a smaller working end-to-end implementation over multiple incomplete features.

9. Expected Outcome

By completing Case Study 3, teams should demonstrate how an existing multi-agent assistant can be evolved into a system that collaborates across agents, remembers useful knowledge, manages prompts centrally, and measures quality through repeatable evaluations. The final solution should make the Case Study 2 architecture easier to improve, observe, and validate.

Submission deadline	Monday, 16 November 2026
Evaluation session	Monday, 16 November 2026, 3:00 PM (same day as the submission deadline)
Submission location	SharePoint folder (link shared with all teams)

Confidential - For Company Internal Use Only